

Instructor Handout 5: Addressing Modes, CPU Organization, and Bus Structures

PCC CS-401: Computer Organization and Architecture

August 7, 2026

Instructor Copy — Not for Student Distribution

5.1 Before You Teach This Lecture

- **Mano, *Computer System Architecture*, 3rd ed., Sec. 8-5 “Addressing Modes,” pp. 256–266** (indexed, page-cited). Read the book’s own worked numerical example (Fig. 8-7) — it gives you a second, book-native worked instance to fall back on if a student’s question needs a different set of numbers than the ones in the slides or below.
- **Mano, Sec. 8-2 “General Register Organization,” pp. 242–246** (indexed). This is the section Fig. 8-2 (“Registers with common ALU”) actually comes from — read it once so you can explain SELA/SELB/SELD confidently if a student asks where those labels in the three-bus diagram come from.
- **Mano, Fig. 5-4, p. 129** (indexed at the figure level; the surrounding Sec. 5-2 is not yet fully indexed). This is the real single-bus/common-bus-system figure the lecture’s single-bus datapath is styled on.
- **Hamacher, *Computer Organization*, Ch. 7** — *not yet page-indexed for this course* (the supplied PDF has no text layer). Cite it only at chapter level with students; do not improvise a page number.

5.2 Extra Worked Examples

These are additional instances beyond the one worked example each topic already has in the student Notes — use them for board practice, a quiz, or to answer “can you do a different one” on the spot.

5.2.1 Addressing Modes: Autoincrement

ADD R1, (R3)+ with $[R3] = 500$ and an operand size of 4 bytes.

1. The mode is autoincrement, so $EA = [R3]$ *before* the increment: $EA = 500$.
2. The operand is fetched from $M[500]$ and used in the ADD.
3. *As a side effect*, R3 is then incremented by the operand size: $[R3] \leftarrow 500 + 4 = 504$.

Instructor note: the point worth drawing out live is that the increment happens *after* EA is used, not before — students conflate autoincrement with pre-increment addressing (which this course does not cover, but which students sometimes assume by analogy to `++i` vs. `i++` in C).

5.2.2 Single-Bus: A Different Register Pair

Trace $R7 \leftarrow R0 + R1$ on the single-bus datapath.

$$T_1 : Y \leftarrow [R0], \quad T_2 : Z \leftarrow [Y] + [R1], \quad T_3 : R7 \leftarrow [Z].$$

Still **3 control steps** — worth stating explicitly that the register *names* never affect the step count; only the operation’s shape (two operands, one ALU op, one result) does.

5.2.3 Three-Bus: A Subtraction

Trace $R2 \leftarrow R6 - R7$ on the three-bus datapath: $A = [R6]$, $B = [R7]$, $C = \text{result}$, ALU set to subtract, all in **1 control step**. Useful for pre-empting the (common) assumption that the three-bus organization’s speed only applies to addition.

5.3 Teaching Notes

These are instructional suggestions grounded in this lecture’s own content and general teaching craft — not facts to be tested, and not independently citation-checked the way the worked examples and reading list above are. Adapt freely to your own classroom.

5.3.1 Addressing Modes

- **Common misconception:** students frequently conflate indirect addressing (pointer in *memory*) with register-indirect addressing (pointer in a *register*) — both “follow a pointer,” but only indirect costs two memory reads. Ask a cold-call question early: “which one is cheaper, and why?”
- **Board work:** draw the indirect-addressing chain (instruction $\rightarrow M[A] \rightarrow M[B] \rightarrow \text{operand}$) live, one box at a time, rather than showing it pre-drawn — the two-hop structure lands better when built incrementally.
- **Live question:** “What would autoincrement addressing look like if it took two separate instructions instead of one?” (Answer: a register-indirect load, followed by a separate increment instruction — this motivates *why* the side-effect mode exists: it saves an instruction per loop iteration.)

5.3.2 Single-Bus Organization

- **Common misconception:** students often read Y and Z as arbitrary extra complexity rather than a *forced consequence* of the one-bus constraint. Before introducing them, ask: “the ALU needs two operands this cycle, but the bus can carry one — where does the first operand wait?” and let the answer motivate Y before you name it.
- **Board sequence:** draw the common bus and its four top registers first; add Y/ALU/Z *below* only after the “where does it wait” question has been asked — drawing the whole diagram at once tends to make Y/Z look decorative rather than necessary.
- **Pacing:** this is the derivation most sections need a second pass on. Consider a fast first pass through the 3-step trace, then a slower second pass where you ask the class to supply each T_i line themselves.

5.3.3 Multiple-Bus Organization

- **Common misconception:** students sometimes attribute the three-bus design's speed to "a faster ALU" rather than to parallel *read* paths. Explicitly contrast: same ALU, same clock, different wiring.
- **Live question:** "Why not build a CPU with ten buses instead of three?" before showing the corresponding slide — most classes converge on "more wiring cost" unprompted, which sets up the design-space-limits discussion well.